

基于MiniFly的无人机编队系统

刚上大二那个暑假，我加入了我们学院的物联网实验室，老师给的题目就是无人机，经过调研，我们选择了正点原子的MiniFly 可二次开发的开源四旋翼无人机，从此项目开始研究，随后我们实现了：

- 1、一台遥控器同步控制多台无人机
- 2、一台遥控器分布控制多台无人机
- 3、使用电脑的串口上位机控制无人机
- 4、基于UWB的室内定位无人机编队

一台遥控器同步控制多台无人机

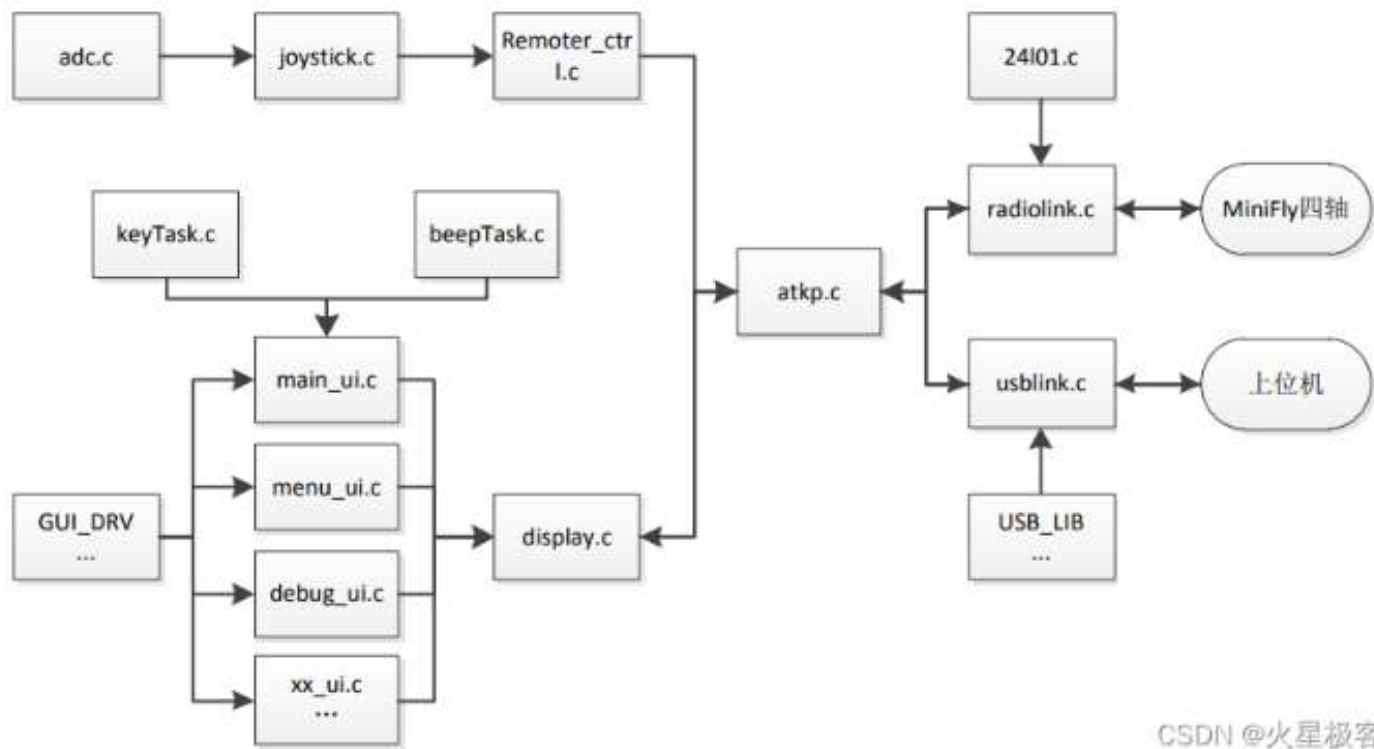
以下将对“一台遥控器同步控制多台无人机”简称为“一控多同步”。
一控多同步，即一台遥控器可以同时控制多台无人机进行起降、俯仰、横滚、航向。

无线通讯方面

要想实现一控多同步，首先要解决遥控信号通讯的问题，我们知道，初始的遥控无人机是一台遥控器控制一台无人机，那么我们就必须改变这个设定，使遥控器在宏观上能够与多台无人机通讯。实现这一目的有两种方式，遥控器处于广播模式或者遥控器实时跳频。
无线电广播的优势是无需跳频，但缺点是单向通信，而MiniFly是双向通信的，如果要利用广播实现一控多同步，那么遥控将损失无人机的回传信息，因此不做考虑。
遥控器跳频可以实现一控多同步，但目前还没实现自组网，因此缺点就是控制多台得修改代码配置。

遥控代码部分

首先我们必须知道遥控器的代码框架



经过研究代码，我发现控制无人机通信的信道和地址的配置参数变量在config_param.c中的configParam中，其中的configParam.radio.channel储存着无线通信的信道，configParam.radio.addressHigh和configParam.radio.addressLow分别储存着地址的高位和低位，而地址就储存在RADIO_ADDRESS这个常量中

```
21
22 configParam_t configParam; /*配置参数全局变量*/
23
24 static bool isInit;
25 static bool isConfigParamOK;
26
27 static configParam_t configParamDefault=
28 {
29     .version = VERSION,
30     .language = DISPLAY_LANGUAGE,
31
32     .radio.number = FIRST_PLANE,
33     .radio.channel = RADIO_CHANNEL1,
34     .radio.dataRate = RADIO_DATARATE,
35     .radio.addressHigh = ((uint64_t)RADIO_ADDRESS1 >> 32),
36     .radio.addressLow = (RADIO_ADDRESS1 & 0xFFFFFFFFFULL),
37 }
```

CSDN @火星极客

因此我决定多添加几个信道和地址，并在结构体中新增了一个radio.number的数据，用于标记这一信道和地址是属于哪一台无人机。最终添加了四台无人机的信道和地址信息，此处我在跳频中只切换地址即可实现多台无人机的分时控制。

```
36 //RADIO_CHANNEL通道数为125(0 - 124)
37 //一个通道可使用6个地址
38 #define ALL_PLANE 0
39
40 #define FIRST_PLANE 1
41 #define RADIO_CHANNEL1 2
42 #define RADIO_ADDRESS1 0x123456789AULL //高四位二进制(10010) 十进制(18)
43
44 #define SECOND_PLANE 2
45 #define RADIO_CHANNEL2 2
46 #define RADIO_ADDRESS2 0x133456789AULL //高四位二进制(10011) 十进制(19)
47
48 #define THIRD_PLANE 3
49 #define RADIO_CHANNEL3 2
50 #define RADIO_ADDRESS3 0x143456789AULL //高四位二进制(10100) 十进制(20)
51
52 #define FOURTH_PLANE 4
53 #define RADIO_CHANNEL4 2
54 #define RADIO_ADDRESS4 0x153456789AULL //高四位二进制(10101) 十进制(21)
55
```

CSDN @火星极客

经此，我们只需要改变configParam.radio.channel、addressHigh和addressLow即可切换遥控控制不同的无人机，因此我将MiniFly中用于翻滚的按钮——KEY_R_SHORT_PRESS用于信道和地址的切换，并使用switch Q语句改变信道和地址。

```

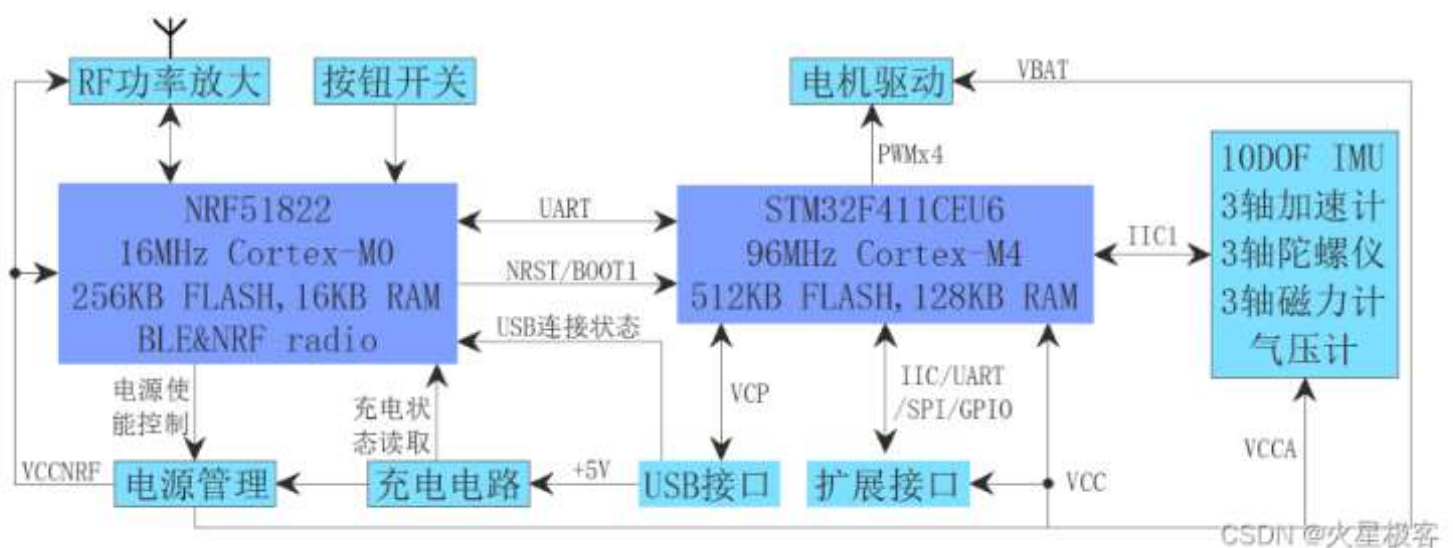
84 int change_plane(void) {
85     switch(sortie) {
86         case 0: configParam.radio.number = ALL_PLANE;
87                 break;
88         case 1: configParam.radio.number = FIRST_PLANE;
89                 configParam.radio.channel = RADIO_CHANNEL1;
90                 configParam.radio.addressHigh = ((uint64_t)RADIO_ADDRESS1 >> 32);
91                 configParam.radio.addressLow = (RADIO_ADDRESS1 & 0xFFFFFFFFFULL);
92                 break;
93         case 2: configParam.radio.number = SECOND_PLANE;
94                 configParam.radio.channel = RADIO_CHANNEL2;
95                 configParam.radio.addressHigh = ((uint64_t)RADIO_ADDRESS2 >> 32);
96                 configParam.radio.addressLow = (RADIO_ADDRESS2 & 0xFFFFFFFFFULL);
97                 break;
98         case 3: configParam.radio.number = THIRD_PLANE;
99                 configParam.radio.channel = RADIO_CHANNEL3;
100                configParam.radio.addressHigh = ((uint64_t)RADIO_ADDRESS3 >> 32);
101                configParam.radio.addressLow = (RADIO_ADDRESS3 & 0xFFFFFFFFFULL);
102                break;
103         case 4: configParam.radio.number = FOURTH_PLANE;
104                 configParam.radio.channel = RADIO_CHANNEL4;
105                 configParam.radio.addressHigh = ((uint64_t)RADIO_ADDRESS4 >> 32);
106                 configParam.radio.addressLow = (RADIO_ADDRESS4 & 0xFFFFFFFFFULL);
107                 break;
108     }
109     uint64_t addr = (uint64_t)configParam.radio.addressHigh<<32 | configParam.radio.addressLow;
110     nrf_setAddress(addr);
111     nrf_setChannel(configParam.radio.channel);
112     sortie_change = sortie; //使改变的标记与当前的架次统一
113     return sortie_change;
114 }

```

CSDN @火星极客

无人机通信代码部分

经过开发指南 [\[4\]](#)，我们得知无人机的通信时由NRF51822负责的



CSDN @火星极客

因此我们只需要修改其中的代码，即可实现手动适配信道和地址。同样的，我通过搜索关键字 [发现](#) 其配置参数放在 config_param.h 中，通过改变地址即可实现多台无人机的信道和地址配置。

```
19 /* 默认配置参数 */
20 #define VERSION 11 /*11表示V1.1*/
21
22 #define RADIO_CHANNEL 2
23 #define RADIO_DATARATE esbDatarate250K
24 #define RADIO_ADDRESS 0x133456789AULL
25 /*
26 #define RADIO_ADDRESS1 0x123456789AULL//高四位二进制（10010） 十进制（18）
27 #define RADIO_ADDRESS2 0x133456789AULL//高四位二进制（10011） 十进制（19）
28 #define RADIO_ADDRESS3 0x143456789AULL//高四位二进制（10100） 十进制（20）
29 #define RADIO_ADDRESS4 0x153456789AULL//高四位二进制（10101） 十进制（21）
30 */
31
32 #define RADIO_ADDRESS4 0x153456789AULL//高四位二进制（10101） 十进制（21）
33
```

至此，即可通过按下右边的按钮切换控制无人机，但要实现同步控制还需要遥控器在选择控制所有无人机的时候处于循环跳频状态。

```
280 //-----飞机架次
281 else if(keyState == KEY_R_SHORT_PRESS)//短按选择飞机架次
282 { //如果当前处于控制所有飞机的状态，即改为控制某一架飞机，从第一架开始计数
283     if(sign_all_plane == true){
284         sign_all_plane = false;
285         control = 1;//需要控制的单一架次
286     }
287     else if(control < PLANE_NUM){//递增
288         control ++;
289     }
290     else{//递增到最后一台，继续控制所有飞机
291         control = 0;//0代表控制所有飞机
292         sign_all_plane = true;
293     }
294     //runBeepAcktion(CANFLY_BEEP);
295 }
```

一台遥控器分布控制多台无人机

以下将对“一台遥控器分布控制多台无人机”简称为“一控多分布”。

一控多分布，即一台遥控器可以同时控制多台无人机进行起降、俯仰、横滚、航向，并且可以在多台无人机飞行过程中单独控制一台无人机而其他保持静止。

无线通讯部分与上面的一控多同步类似，只是修改了跳频部分，需要不停的跳频。

```
322 //-----
323 //控制所有的飞机即从1~PLANE_NUM不断循环
324 //当处于控制所有飞机以及处于非瞬时性任务控制的时候，不断跳频//sign_all_plane == true &&
325 //无论如何一直跳频
326 if(task_in_progress == false){
327     if(sortie < PLANE_NUM)
328         sortie ++;
329     else
330         sortie = 1;
331     //sortie = 1;
332     send_sortie = change_plane();
333     delay_ms(15); //延迟防止宕机
334
335     atkp_t p;
336     p.msgID = UP_SEND;
337     p.dataLen = 1;
338     p.data[0] = send_sortie;
339     usblinkSendPacket(&p); //将atkp包发送
340 }
```

CSDN @火星极客

无人机控制部分

之后就需实现对某一台无人机进行单独控制，具体代码在remoter_ctrl.c中，当校验到控制的架次与当前频段和地址匹配时，发送遥控控制数据，否则发送悬停数据。

```
else{//遥控控制
    if(getformation() == false){ //表示当前处于编队的单机控制状态
        //当需要控制的飞机等于遥控射频的飞机架次
        if(getformationcontrol() == getformationplane()){
            //单独遥控控制
            send.thrust = flydata.thrust; //高度
            send.pitch = flydata.pitch; //前进
            send.roll = flydata.roll; //横滚
            send.yaw = flydata.yaw; //航向
        }
        else{
            send.thrust = 50; //高度
            send.pitch = 0; //前进
            send.roll = 0; //横滚
            send.yaw = 0; //航向
        }
    }
}
```

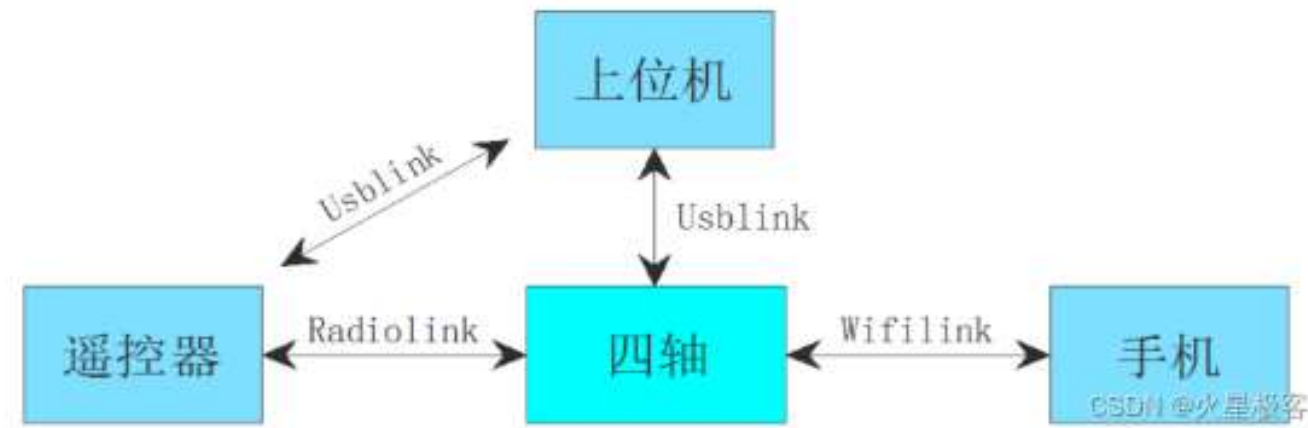
CSDN @火星极客

至此，即可实现一台遥控器在控制多台无人机做编队飞行，并且在集群飞行过程中单独对某一台进行位置调整。

使用电脑的串口上位机控制无人机

要想实现无人机编队飞行，单靠人工的手动飞行是万万不能达到要求的，因此我们必须考虑使用电脑来控制无人机的飞行，那么最简单的方式就是电脑利用遥控器控制无人机，那么我们就需要发送指令到遥控器上，即串口通信。

MiniFly 遥控器、四轴主机、上位机之间通信链路关系图如下图 2.1.1 所示：



由MiniFly的遥控器开发指南可知，上位机控制遥控器的函数是在Usblink.c中，而且原函数中就有开发出这一功能，我只是适当拓展，将对收到的某一个我设定的帧做出我想要的反应。

遥控串口通信接收指令

具体修改如下

```
222 case waitForChksum1:
223     /*
224     memset(str, 0, 22);
225     sprintf(str, " cksum:%3d      c:%3d", cksum, c);
226     oled_showString(0, 35 + dataIndex*9, (u8*)str, 6, 8);
227     //-----*/
228     if (cksum == c)/*所有校验正确*/
229     {
230         if(control_data.msgID == DOWN_FLY)
231             rx_fly_data = true;
232         else
233             rx_control_data = true;
234         xQueueSend(rxQueue, &rxPacket, 0);
235     }
236     else
237     {
238         rxState = waitForStartByte1;
239     }
240     rxState = waitForStartByte1;
241     break;
242 default:
243     break;
244 }
```

将校验正确后的指令做个分类，如果是我发送的DOWN_FLY的指令，那么会置标志位为真。

```
253 bool getdata(atkp_t *p) {
254     int i;
255     if(rx_control_data) { //通过校验值
256         (*p).msgID = control_data.msgID;
257         (*p).dataLen = control_data.dataLen;
258         for(i = 0; i < control_data.dataLen; i++) {
259             (*p).data[i] = control_data.data[i];
260         }
261         rx_control_data = false;
262         return true;
263     }
264     return false;
265 }
266 bool get_flydata(atkp_t *p) {
267     int i;
268     if(rx_fly_data) { //通过校验值
269         (*p).msgID = control_data.msgID;
270         (*p).dataLen = control_data.dataLen;
271         for(i = 0; i < control_data.dataLen; i++) {
272             (*p).data[i] = control_data.data[i];
273         }
274         rx_fly_data = false;
275         return true;
276     }
277     return false;
278 }
```

CSDN @火星极客

从而驱动指令数据去到需要他的地方，比如起飞的指令数据：

```
165 get_flydata(&flight_land); //获取串口信息
166 if(flight_land.msgID == DOWN_FLY) {
167     if(flight_land.data[0] == 0 && isMFCanFly == true) { //达到飞行条件以及获取了串口指令
168         oled_showString(30, 17, (u8*)"COM FLIGHT", 6, 8);
169         rx_flight = true; //指令一键起飞标记，无论哪一架，或者是所有
170         RCLock = false; //解锁
171     }
172     else {
173         oled_showString(30, 17, (u8*)"          ", 6, 8);
174         rx_stop = true; //指令一键停机标记
175         RCLock = true;
176     }
177     flight_land.msgID = 0x00; //清空防止再次调用
178 }
```

CSDN @火星极客

或者是姿态飞行的指令数据

```
203 //
204 else if(rx_data.msgID == DOWN_DATA){ //串口指令控制
205     oled_showString(30, 17, (u8*)"COW CONTROL", 6, 8); //显示串口控制
206
207     control_planenum = rx_data.data[4]; //获取控制的架次并进行赋值
208     if(control_planenum == 0) { //当收到控制所有飞机指令时，将指令覆盖到所有指令集中
209         for(int i = 0; i <= PLANE_NUM; i++){
210             //control_planenum == 0是代表所有架次，即发送时直接无差别发送同一组数据
211             control_plane[i].thrust = rx_data.data[0];
212             control_plane[i].pitch = rx_data.data[1] - 50; //-50为设置中位
213             control_plane[i].roll = rx_data.data[2] - 50;
214             control_plane[i].yaw = rx_data.data[3] - 50;
215         }
216     }
217     else { //将需要控制的某一架次的数据赋值
218         control_plane[control_planenum].thrust = rx_data.data[0];
219         control_plane[control_planenum].pitch = rx_data.data[1] - 50;
220         control_plane[control_planenum].roll = rx_data.data[2] - 50;
221         control_plane[control_planenum].yaw = rx_data.data[3] - 50;
222     }
223     //当最后一次发送的控制架次数据不是0，即代表处于跳频分别控制单机状态
224     if(control_planenum != 0){
225         //当前遥控射频的飞机架次赋予控制的架次
226         control_planenum = getformationplane();
227         //因为send是不停发送的，所以每次调用getformationplane时候控制的飞机应该都不同
228         //再将其对应的指令发送
229     }
230     send.thrust = control_plane[control_planenum].thrust; //高度
231     send.pitch = control_plane[control_planenum].pitch; //前进
232     send.roll = control_plane[control_planenum].roll; //横滚
233     send.yaw = control_plane[control_planenum].yaw; //航向
234     /*
```

最后就是需要 we 使用正确的指令去发送给遥控器，这需要 we 自行定义，我个人是这样定义的

控制指令解释

功能	第一位	第二位	第三位	第四位	第五位	第六位	第七位	第八位	第九位	第十位
前进	AA	AF	06	05	32	44	32	32	00	3E
停止	AA	AF	06	05	32	32	32	32	00	2C
降落	AA	AF	06	05	19	32	32	32	00	13
解释	起始位	起始位	下行指令	数据长度	油门	俯仰	横滚	航向	架次	校验位

第一、二位为固定的起始标记位。

第三位为自行设置的下行指令，06 是普通控制的指令集，04 是一键起飞、降落与急停的指令集。

第四位为数据长度，即第五到第九有五个数据。

第五到第八位是指令范围，十进制控制范围 0-100,50 为中位，需要转换为十六进制，即 50 的十六进制为 32。

第九位代表飞机的架次；00 代表所有飞机，01 表示控制第一架，02 表示控制第二架……

第十位是第一位到第九位数据的和，再除 256 的余数。转换成十六进制。

CSDN @火星极客

功能	第一位	第二位	第三位	第四位	第五位	第六位	第七位
一键起飞	AA	AF	04	02	00	00	5F
紧急停机	AA	AF	04	02	01	00	60

第一、二位为固定的起始标记位。

第三位为自行设置的下行指令，06 是普通控制的指令集，04 是一键起飞、降落与急停的指令集。

第四位为数据长度，即第五到第六有两个数据。

第五位是控制的指令，00 为起飞与降落，01 为紧急停机。

第六位代表飞机的架次；00 代表所有飞机，01 表示控制第一架，02 表示控制第二架……

第七位是第一位到第六位数据的和，再除 256 的余数。转换成十六进制。

CSDN @火星极客

至此，我们只需要在电脑上打开串口助手，发送控制指令，就可以控制无人机的飞行。当发送的指令足够快，就可以组成一场无人机的编队表演。